



Designing the instruction set of your custom CPU

You've got a memory map. You know where your code lives, where your stack lives, and how to talk to hardware. Now comes the part that determines what your CPU can actually **do**: the instruction set.

This document walks through every decision behind the instruction set of **MyCustomISA**. As with the memory map, nothing here is arbitrary. Every choice is the result of a tradeoff, and understanding the tradeoffs is more important than memorising the encoding.

Step 1: pick an operand order

Before any bit-twiddling, you need to decide a piece of syntactic convention that will follow you forever: **in an instruction with two operands, which one comes first — the source or the destination?**

There are two camps in the real world:

- **Intel syntax** (x86, ARM): destination first. `MOV R3, R1` means "put R1 into R3."
- **AT&T syntax** (GCC on x86, many RISC assemblers): source first. `MOV R1, R3` means "move R1 to R3."

Neither is objectively right. The destination-first camp argues that it mirrors how assignment works in most programming languages (`x = 5` puts the target on the left). The source-first camp argues that it reads like a natural sentence: "take this value, put it there." Both are defensible.

Decision: MyCustomISA uses source-first, AT&T-style. It reads like an arrow flowing left to right:

```
MOVE R1, R3      ; R1 → R3
ADD  R1, R3      ; R3 = R3 + R1
```

Once you pick a convention, you must be **absolutely consistent**. Every instruction, forever,

follows the same rule.

Step 2: count the bits

You're going to discover early on that there's never enough room. A 16-bit instruction has exactly 16 bits to encode everything: what to do, where to read from, where to write to. Let's see how quickly that budget runs out.

Take a simple instruction: `LOAD [0xABCD], R3`. How many bits does it need?

- The opcode (what this instruction does): a handful of bits
- A 16-bit memory address: 16 bits, the full width of your address space
- A register identifier (you have 16 registers): 4 bits

```
opcode + 16-bit address + 4-bit register = 20+ bits minimum
```

You've already overshot 16 bits, and you haven't even allocated a single bit to the opcode yet.

This is the fundamental tension of ISA design: **rich instructions don't fit in narrow words**. You need a way out.

Step 3: variable-length instructions

The cleanest way out is to accept that **not all instructions need to be the same length**. Some are simple — they only refer to registers, which are cheap to encode. Some need to embed a full 16-bit address. So:

- **16-bit instructions** for operations that only touch registers
- **32-bit instructions** for operations that need a 16-bit payload (immediate value or absolute address)

This is exactly what x86 does (with even more length variants), and what ARM Thumb-2 does.

But variable length introduces a problem you didn't have before: when the CPU fetches the first 16 bits of an instruction, **how does it know whether to fetch another 16 bits or not?**

The answer is going to fall out naturally from the next decision — but keep this question in

mind.

Step 4: the opcode explosion problem

Before fixing the encoding, you need to figure out what kinds of operands an instruction can take. Consider MOVE:

```
MOVE R1, R3      ; register to register
MOVE R1, [R3]    ; register to memory at address held in R3
MOVE R1, [0xABCD] ; register to memory at a literal address
MOVE [0xABCD], R3 ; memory at a literal address to register
MOVE 42, R3      ; immediate value to register
...

```

If each combination needs its own opcode (`MOVE_RR` , `MOVE_RM` , `MOVE_RI` , ...), your opcode space explodes. And the same problem repeats for every instruction that can take multiple operand forms: ADD, SUB, AND, OR, CMP...

So here's the key insight, the same one that powers the Motorola 68000:

Instead of encoding "what the instruction does" and "what kind of operands it takes" in the same bits, **separate them**.

The opcode says **what to do**. A separate field — the **addressing mode** — says **how to interpret each operand**. Now `MOVE` has one opcode, and the mode bits tell the CPU whether to grab a register, follow a pointer, or read a literal.

Step 5: define the addressing modes

The 68000 has about ten addressing modes — pre-decrement, post-increment, PC-relative with displacement, and so on. We don't need all of that. For MyCustomISA v1, four modes cover every realistic use case:

Mode	Binary	Syntax	Meaning
Immediate	<code>00</code>	<code>42 ,</code>	A 16-bit literal value embedded in the

Mode	Binary	Syntax	Meaning
		0xABCD	instruction
Register	01	R3	The value held in a register
Indirect register	10	[R3]	The value at the memory address held in a register
Indirect address	11	[0xABCD]	The value at a 16-bit absolute memory address

Four modes fit in 2 bits — a tiny price to pay for the encoding cleanliness we get.

And notice the elegance: **modes 00 and 11 carry a 16-bit payload, modes 01 and 10 don't**. So the mode field alone tells the CPU whether the instruction is 16 or 32 bits long. We don't need a separate length discriminator — it's implied by the modes.

Step 6: load-store architecture

Now think about the worst case for a two-operand instruction:

```
MOVE [0xAAAA], [0xBBBB] ; memory to memory
```

This would need two 16-bit addresses embedded in the instruction — 32 bits of payload alone, on top of the opcode and mode fields. That's a 48-bit instruction, breaking our nice 16/32 split.

There's an elegant principle from RISC architectures that solves this:

You can't move data directly between two memory locations. All memory access goes through registers.

If you want to copy memory to memory, you do it in two instructions: read into a register, write from the register. This is called a **load-store architecture**, and it's how RISC-V, ARM, and MIPS all work.

In MyCustomISA, this becomes three specialised instructions:

- `LOAD addr, R` — read from memory into a register

- `STORE R, addr` — write from a register into memory
- `MOVE R, R` — register to register only

Each has a fixed destination type baked into the opcode itself, so the CPU doesn't need a `mode_d` field. That saves 2 bits in the encoding — bits you absolutely need.

Step 7: list the instructions

Before designing the encoding any further, you need to know what you're encoding. Here is the full instruction set, grouped by category:

Category	Instructions
Data movement	LOAD, STORE, MOVE
Arithmetic	ADD, SUB, MUL, DIV, SHL, SHR
Logic	AND, OR, NOT
Comparison & branching	CMP, JMP, JE, JNE, JGT, JGE, JLT, JLE, JA, JAE, JB, JBE
Subroutine	CALL, RET
Stack	PUSH, POP, GETSP, SETSP
Interrupt	MVINT, TRAP, RETI
Flag manipulation	SEZ, CLZ, SEC, CLC, SEV, CLV, SEN, CLN
Control	HALT

On signed vs. unsigned jumps. Notice the four `Jxx` and four `Jxxx` variants. Comparing `0xFFFF` to `0x0000` gives different answers depending on whether you treat them as unsigned (65535 vs 0 — first is bigger) or signed (-1 vs 0 — first is smaller). MyCustomISA needs both interpretations, so jumps come in two flavours: `JGT/JLT/...` for signed and `JA/JB/...` for unsigned. The CPU sets both the C and V flags after every arithmetic operation; your program picks which to check.

On GETSP/SETSP. The stack pointer isn't a general-purpose register, but you still need a way to read and write it — for example, to save and restore the entire CPU context. Two dedicated

instructions handle this without polluting the register file.

On flag manipulation. The memory map specified that flags can only be modified one at a time, never as a whole word. So we need a `SET` and `CLEAR` instruction for each of the four user-controllable flags (Z, C, V, N). That's 8 instructions. R is read-only.

On MVINT. This sets an interrupt vector to point to a handler. Originally we tried to pass both as 16-bit addresses, but that's 32 bits of payload alone. Instead, we identify the interrupt by its number (3 bits, enough for our 7 interrupts), with the 16-bit handler address as the second operand. Much cleaner.

That's 36 instructions total. With a 5-bit opcode field we have 32 slots — not quite enough on the face of it. But we have a clever way out.

Step 8: the three instruction formats

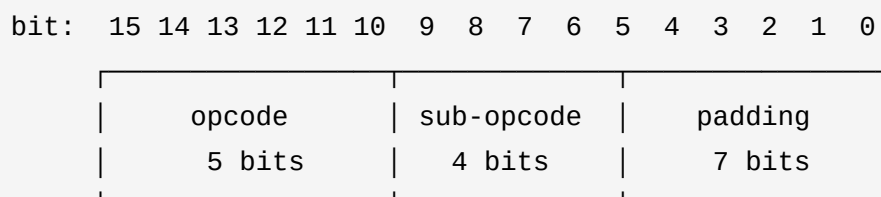
Count operands across the instruction set:

- **No operands:** HALT, RET, RETI, and all 8 flag instructions → 11 instructions
- **One operand:** PUSH, POP, NOT, all jumps, CALL, TRAP, GETSP, SETSP → 18 instructions
- **Two operands:** LOAD, STORE, MOVE, ADD, SUB, MUL, DIV, SHL, SHR, AND, OR, CMP, MVINT → 13 instructions

That's a natural split into three families. So instead of one universal encoding, MyCustomISA has three **instruction formats**, each shaped for its operand count.

Format 0 — no operands

The whole instruction is just an opcode. The remaining bits become a **sub-opcode** to distinguish between many related no-operand instructions:



By giving up just **one** opcode slot (`0x00`) at the primary level, we get 16 sub-slots, of which we use 11 today and keep 5 for future expansion. This is exactly how the 68000 encodes its zero-operand instructions.

Format 1 — one operand

For PUSH, POP, NOT, jumps, CALL, TRAP, GETSP, SETSP.

16-bit version (operand is a register or indirect register):

bit: 15 14 13 12 11 10 9 8 7 6 5 4 3 2 1 0

opcode	mode	reg id	padding
5 bits	2 b.	4 bits	5 bits

32-bit version (operand is an immediate or indirect address):

First word:

bit: 15 14 13 12 11 10 9 8 7 6 5 4 3 2 1 0

opcode	mode	padding
5 bits	2 b.	9 bits

Second word:

16-bit immediate / address

Format 2 — two operands

For arithmetic, logic, comparison, and data movement.

16-bit version (source is a register or indirect register, destination is a register):

bit: 15 14 13 12 11 10 9 8 7 6 5 4 3 2 1 0

opcode	mode	reg_s	reg_d	p
5 bits	2 b.	4 bits	4 bits	1b

32-bit version (source is an immediate/indirect address, or destination is an indirect address):

First word:

bit: 15 14 13 12 11 10 9 8 7 6 5 4 3 2 1 0

opcode	mode	reg	padding
5 bits	2 b.	4 bits	5 bits

Second word:

16-bit immediate / address

For **LOAD**, the second word holds the source address and **reg** is the destination register. For **STORE**, the second word holds the destination address and **reg** is the source register. For **MVINT**, the second word holds the handler address and **reg** (only 3 bits used) identifies the interrupt.

Step 9: identifying the format

When the CPU fetches an instruction, it sees 16 bits and needs to know:

1. Which format is this?
2. How many more bytes do I need to fetch?

The answer to both falls out of the **opcode value**. We assign opcodes in clustered ranges so the CPU can identify the format with a simple range check:

Range	Format
0x00	Format 0 (with sub-opcode in the lower bits)
0x01 — 0x12	Format 1
0x13 — 0x1F	Format 2

Once the format is known, the **mode field** tells the CPU whether the instruction is 16 or 32 bits — no separate length flag needed.

Step 10: the complete opcode map

Putting it all together:

Format 0 — 0x00 with sub-opcode

Sub-opcode	Mnemonic	Description
0x0	HALT	Halt the CPU and freeze its state.
0x1	RET	Return from subroutine. Pops PC from the stack.
0x2	RETI	Return from interrupt. Restores the full saved context.
0x3	SEZ	Set the Zero flag.
0x4	CLZ	Clear the Zero flag.
0x5	SEC	Set the Carry flag.
0x6	CLC	Clear the Carry flag.
0x7	SEV	Set the Overflow flag.
0x8	CLV	Clear the Overflow flag.
0x9	SEN	Set the Negative flag.
0xA	CLN	Clear the Negative flag.
0xB — 0xF	—	Reserved for future use.

Format 1 — 0x01 – 0x12

Opcode	Mnemonic	Valid modes	Description
0x01	PUSH	immediate, register, indirect register, indirect address	Push the operand onto the stack.
0x02	POP	register, indirect register, indirect address	Pop the top of the stack into the operand.
0x03	NOT	register	Bitwise complement of the register, in place.
0x04	JMP	immediate, indirect register, indirect address	Unconditional jump.
0x05	JE	(same)	Jump if equal (Z = 1).
0x06	JNE	(same)	Jump if not equal (Z = 0).
0x07	JGT	(same)	Jump if greater than, signed (Z = 0 AND N = V).
0x08	JGE	(same)	Jump if greater or equal, signed (N = V).
0x09	JLT	(same)	Jump if less than, signed (N ≠ V).
0x0A	JLE	(same)	Jump if less or equal, signed (Z = 1 OR N ≠ V).
0x0B	JA	(same)	Jump if above, unsigned (C = 0 AND Z = 0).
0x0C	JAЕ	(same)	Jump if above or equal, unsigned (C = 0).
0x0D	JB	(same)	Jump if below, unsigned (C = 1).
0x0E	JBE	(same)	Jump if below or equal,

Opcode	Mnemonic	Valid modes	Description
			unsigned (C = 1 OR Z = 1).
0x0F	CALL	immediate, indirect register, indirect address	Push PC to stack, jump to operand.
0x10	TRAP	immediate	Trigger a user-defined trap. Operand is the trap number.
0x11	GETSP	register	Copy the value of SP into the register.
0x12	SETSP	register	Copy the value of the register into SP.

Format 2 — 0x13 – 0x1F

Opcode	Mnemonic	Source modes	Destination	Description
0x13	LOAD	immediate, indirect register, indirect address	register	Read value from source, write to register.
0x14	STORE	register	indirect register, indirect address	Write register value to destination memory location.
0x15	MOVE	register	register	Copy source register to destination register.
0x16	ADD	register, immediate	register	dest = dest + src. Sets Z, C, V, N.
0x17	SUB	register, immediate	register	dest = dest – src. Sets Z, C, V, N.
0x18	MUL	register, immediate	register	dest = dest × src. Sets Z, C, V, N.

Opcode	Mnemonic	Source modes	Destination	Description
0x19	DIV	register, immediate	register	dest = dest ÷ src. Sets Z, N. Triggers IVT_DIVIDE_BY_ZERO if src = 0.
0x1A	SHL	register, immediate	register	Shift dest left by src bits. Sets Z, C, N.
0x1B	SHR	register, immediate	register	Shift dest right by src bits. Sets Z, C, N.
0x1C	AND	register, immediate	register	Bitwise AND. Sets Z, N.
0x1D	OR	register, immediate	register	Bitwise OR. Sets Z, N.
0x1E	CMP	register, immediate	register	Computes dest – src, sets flags, discards the result.
0x1F	MVINT	interrupt number (3 bits) + immediate handler address	—	Set the IVT register for the given interrupt to the given handler address.

Step 11: worked encoding examples

A few concrete cases to sanity-check the format.

HALT (Format 0)

Primary opcode 0x00 = 00000, sub-opcode 0x0 = 0000
 Encoded: 00000 0000 00000000 → 0x0000 (16 bits)

CLC (Format 0)

Primary opcode 0x00 = 00000, sub-opcode 0x6 = 0110
Encoded: 00000 0110 0000000 → 0x0300 (16 bits)

PUSH R5 (Format 1, 16-bit)

Opcode 0x01 = 00001, mode = register = 01, reg = R5 = 0101
Encoded: 00001 01 0101 00000 → 0x0AA0 (16 bits)

JMP [0xABCD] (Format 1, 32-bit)

Opcode 0x04 = 00100, mode = indirect address = 11
First word: 00100 11 0 0000 0000 → 0x26C0
Second word: 1010 1011 1100 1101 → 0xABCD
Encoded: 0x26C0 0xABCD (32 bits)

ADD R1, R3 (Format 2, 16-bit)

Opcode 0x16 = 10110, mode_s = register = 01, reg_s = R1 = 0001, reg_d = R3 = 0011
Encoded: 10110 01 0001 0011 0 → 0xB226 (16 bits)

STORE R1, [0xABCD] (Format 2, 32-bit)

Opcode 0x14 = 10100, mode = indirect address = 11, reg = R1 = 0001
First word: 10100 11 0001 00000 → 0xA620
Second word: 1010 1011 1100 1101 → 0xABCD
Encoded: 0xA620 0xABCD (32 bits)

What comes next

With the instruction set locked down, you now have everything needed to start writing your **assembler in Rust**. The assembler's job is to take human-readable text like:

```
LOAD [0xABCD], R3  
ADD  R1, R3  
STORE R3, [0xABCF]
```

...and produce the exact 16-bit and 32-bit words that this document describes. Labels get resolved to addresses, mnemonics get translated to opcodes, and operands get encoded with the right mode bits.

That's where MyCustomISA stops being a design exercise and starts being a real toolchain.

MyCustomISA — a custom 16-bit ISA for FPGA and virtual machine implementation.